# PROOF OF AI

## Native Execution Proofs and the Hamiltonian Market Maker for Verified Compute

Zach Kelling

*Hanzo AI*

June 2026

**Abstract**

A decentralized AI network must do two things a token network does not: it must *prove* that operators performed the AI work they are paid for, and it must *price* that work into a coin without a trusted oracle. We give a complete treatment of both, joined at a single seam.

For the proof: a digital signature proves *authorship* of a message; a plurality vote proves *agreement* among reporters; neither proves *computation* — that the claimed model was run on the claimed input to produce the claimed output. The cheapest robbery of such a network is therefore a one-bit modal guess that joins the quorum and is minted alongside honest work. We present Proof of AI (PoAI): the proof is not a separate artifact bolted onto inference, it is emitted *as a side-effect of execution by the Hanzo engine itself*. Because $\sim$95% of a forward pass is matrix multiplication $C = A \cdot B$, each layer is verified in $O(n^2)$ instead of recomputed in $O(n^3)$ using Freivalds' algorithm over a prime field $\mathbb{F}_p$, $p = 2^{61} - 1$, applied to the *exact* `int8` integer accumulator. Integer arithmetic is bit-exact across CPU, GPU, and backends, so the check has *zero* false-reject. PoAI proves that an entire computation *graph* ran — inference, embedding, and training are all graphs, verified by one proof system — and its soundness is information-theoretic, hence post-quantum.

For the price: we develop the **Hamiltonian Market Maker (HMM)**, an automated market maker for the one asset PoAI makes well-defined — a unit of *cryptographically verified* compute, settled in AICoin. We define the market state $(q, p)$ (inventory of proven-compute units and their marginal price), a conserved quantity $H$ (the Hamiltonian, the maker's bonded value), and trades as the canonical flow that holds $H$ on its level set. We prove the properties this buys: *path-independence* (the cost of a trade depends only on its endpoints), *bounded round-trip* (a closed loop costs zero before fees, strictly positive after, so no in-curve arbitrage exists), a *bounded worst-case market-maker subsidy* (the LMSR bound $b \ln N$, recovered as the entropic special case, and the divergence-loss bound for constant-function makers), and a *monotone price–supply law* that ties price to verified-compute supply. We show LMSR (Hanson) and constant-function AMMs (Uniswap's $x \cdot y = k$ is a degenerate conserved quantity) are two potentials of one framework, with HMM the principled Legendre-duality generalization. The supply side is honest by construction: only miners bonded in proportion to declared capacity (`MinerStakeRegistry`) feed the curve, and a network-wide no-double-mint ledger (`AIComputeRegistry`) guarantees each inventory increment is a distinct verified unit.

We state plainly what is live: the verifier core, the typed graph-composition, and the determinism prerequisites ship and are tested in Rust and Go; the chain enforces no-proof-no-mint; end-to-end proof *emission* from a running model is designed and merged but not yet live. The HMM is a protocol-design contribution: every claim is a theorem with a proof, and we report no empirical benchmark we did not measure.

# Contents

**1 Introduction: prove the work, then price it** **3**

**2 The Freivalds core: verifying a forward pass** **5**

**3 Four proofs over one primitive** **6**

**4 From matmuls to a graph: typed composition** **7**

**5 Determinism is the load-bearing property** **8**

**6 Commit now, verify later** **8**

**7 Binding: a proof points at exactly one (model, input, output)** **9**

**8 Floating point: by tier, not by tolerance** **10**

**9 Post-quantum by construction** **10**

**10 Implementation status** **11**

**11 The market for verified compute** **12**

**12 The Hamiltonian formulation** **13**
12.1 State, potential, price . . . . . 13
12.2 The Hamiltonian and the equations of motion . . . . . 14

**13 Conservation laws and the four guarantees** **15**
13.1 Path independence . . . . . 15
13.2 Bounded round-trip and no in-curve arbitrage . . . . . 15
13.3 Bounded market-maker subsidy . . . . . 16
13.4 The monotone price–supply law (the PoAI coupling) . . . . . 17
13.5 Slippage is convex . . . . . 17

**14 LMSR and Uniswap are two potentials of one framework** **17**

**15 Multi-asset HMM over heterogeneous compute** **19**

**16 Binding the curve to verified supply** **20**

**17 Conclusion** **21**

# 1 Introduction: prove the work, then price it

A network that pays operators to run AI faces two questions that are usually answered badly. *Did the operator do the work?* and *what is the work worth?* The first is a soundness question; the second is a market-design question. This paper answers both, and the answers meet at a single object: the *proven-compute unit* (PCU), a quantum of work that has been verified to be real AI computation. Part I (§§2–10) builds the proof that makes a PCU well-defined. Part II (§§11–16) builds the market maker that prices it.

**The proof question.** Three properties are easy to confuse, and only one is the thing that must be secured.

- A **signature** proves *authorship*: this party committed to this message. It says nothing about whether any computation occurred.
- A **plurality** (a quorum, a majority vote over reported outputs) proves *agreement*: $m$ reporters submitted the same answer. It is only as honest as the reporters, and it rewards consensus, not work.
- A **proof of computation** proves the load-bearing fact: this operator ran *this* model on *this* input and obtained *this* output.

The gap between the first two and the third is an economic vulnerability. If minting requires only a signed output that matches the quorum, the cheapest attack is a $\sim$1-bit *modal guess*: an operator that never loads the model, predicts the most likely token (or copies a neighbor), signs it, joins the plurality, and is paid. No matmul is performed; the marginal cost of the "work" is a coin flip. PoAI closes the gap by making the proof a *property of execution* rather than a property of reporting.

**The price question.** Once work is provably real, a decentralized market must convert it into coin. An order book needs matched counterparties and deep liquidity that a young compute market lacks; an oracle reintroduces the trusted third party the proof just removed. Automated market makers (AMMs) solve this for fungible tokens, but a unit of compute is not a token: it is supplied continuously, its supply is gated by a cryptographic proof, and the market maker must subsidize price discovery with a *bounded* worst-case loss the way Hanson's LMSR does for prediction markets. We give the AMM that fits: the Hamiltonian Market Maker, whose conserved quantity ties the AICoin price of compute directly to the supply of *verified* PCUs.

**Why they belong in one paper.** The honesty of the price rests entirely on the honesty of the supply. An AMM that prices unverified "compute" prices a modal guess; the curve is only as sound as the unit feeding it. PoAI is what makes the HMM's inventory axis $q$ mean something: every increment of $q$ is a distinct, verified, non-replayable PCU. Conversely, the HMM is what gives PoAI an economic endpoint: a proof that settles into a fairly-priced coin rather than a flat subsidy. The seam between them is the `MinerStakeRegistry` (bonded supply) and the `AIComputeRegistry` (no double mint), grounded in §16.

**Contributions.**

- **(Part I, proof.)** The framing signature $\neq$ plurality $\neq$ computation and the modal-guess attack it exposes (§1); the Freivalds core over the exact `int8` accumulator in $\mathbb{F}_p$ with zero false-reject, grounded line-for-line in `poi.rs` (§2); the four proofs PoE/PoI/PoT/PoC over one matmul primitive (§3); typed *graph* composition that lifts per-matmul soundness to whole-forward-pass soundness (§4); the determinism prerequisites (§5); the commit-now/verify-later activation-trace MMR with interactive bisection (§6); the `reportData` binding (§7); the by-tier floating-point treatment (§8); the post-quantum argument (§9); and an honest implementation-status section (§10).
- **(Part II, price.)** The Hamiltonian Market Maker: a full phase-space formulation of compute pricing (§12); five proved results — path independence (Thm. 4), bounded round-trip / no in-curve arbitrage (Thm. 5), bounded market-maker subsidy (Thm. 7, recovering LMSR's $b \ln N$), the monotone price–supply law (Thm. 9), and slippage convexity (Prop. 10) — in §13; the demonstration that LMSR and Uniswap are two potentials of the one framework (§14); the multi-asset generalization over heterogeneous compute (§15); and the binding of the curve to PoAI's verified supply through the bond and no-double-mint registries (§16).

## Part I — The Proof

## 2 The Freivalds core: verifying a forward pass

A transformer forward pass is dominated by matrix multiplication: the attention projections, the MLP/expert GEMMs, and the output head are all of the form $C = A \cdot B$, and together they are $\sim$95% of the floating (here, integer) operations. This is the lever. To *recompute* a layer $A \in \mathbb{Z}^{t\times k}$, $B \in \mathbb{Z}^{k\times n}$, giving $C \in \mathbb{Z}^{t\times n}$, costs $O(t\,k\,n)$ — the same work the prover did. To *verify* it, we do not need to redo it.

**Theorem 1** (Freivalds, 1977, over $\mathbb{F}_p$)**.** *Let $p$ be prime, $A, B, C$ integer matrices of conforming shape reduced into $\mathbb{F}_p$, and let $r \in \mathbb{F}_p^n$ be drawn uniformly at random. If $C = A \cdot B$ then $A\,(B\,r) = C\,r$* always. *If $C \neq A \cdot B$ then*

$$\Pr_r\big[A\,(B\,r) = C\,r\big] \;\leq\; \frac{1}{p}.$$

*Computing $B\,r$, then $A\,(B\,r)$, and $C\,r$ costs $O(tk+kn+tn) = O(n^2)$ matrix–vector work, one order below the $O(n^3)$ of the multiply.*

*Proof.* Let $D = C - A \cdot B \neq 0$ over $\mathbb{F}_p$, so some row $D_i \neq 0$. The event $D\,r = 0$ forces $\langle D_i, r\rangle = 0$, a single non-trivial linear constraint on $r$; its solution set is a hyperplane, an affine subspace of codimension one, containing exactly $p^{n-1}$ of the $p^n$ points of $\mathbb{F}_p^n$, i.e. a $1/p$ fraction. An honest $C$ gives $D = 0$ and passes for every $r$, which is why there is no false-reject. □

Running $k$ independent challenge vectors multiplies the per-vector soundness error, giving a tampered $C$ a survival probability of at most $(1/p)^k$.

**Why integers, and why $p = 2^{61} - 1$.** The engine's `int8` quantized path multiplies signed 8-bit operands into a 32-bit accumulator ($i8 \cdot i8 \to i32$, the `f8q8` path in `hanzo-quant`). That accumulator is an *exact integer*; it is identical on every backend that respects the reduction, because integer addition and multiplication carry no rounding. Verifying over $\mathbb{F}_p$ with the Mersenne prime $p = 2^{61} - 1$ keeps every intermediate inside 61 bits, so a single 128-bit multiply (`fmul`) and a conditional subtract (`fadd`) suffice and never overflow. The result is the property the rest of PoAI depends on: **zero false-reject across CPU/GPU/backends**. Soundness error is $1/p \approx 2^{-61}$ per challenge vector; $k = 2$ vectors give $\leq 2^{-122}$.

**The shipped API.** The core lives in `hanzo-engine/src/poi.rs`: dependency-free, backend-agnostic, unit-tested in isolation. The surface is small and is mirrored byte-for-byte by the Go verifier the chain consumes (`luxfi/crypto/poi`, with `NewMat`/`Verify`/`VerifyMulti`/`DeriveChallenges` and the same $p = (1 \ll 61) - 1$).

```
pub const P: u64 = (1u64 << 61) - 1;       // Mersenne prime field modulus

pub struct Mat { rows, cols, data: Vec<i64> }  // exact i32/i64 accumulators

// Verify C == A*B for ONE challenge r:  A*(B*r) == C*r over F_p, O(tk+kn+tn).
pub fn freivalds_verify(a, b, c: &Mat, r: &[u64]) -> bool;

// Verify against k challenges; honest C passes ALL, tampered C fails w.p. >= 1-(1/p)^k.
pub fn freivalds_verify_multi(a, b, c: &Mat, challenges: &[Vec<u64>]) -> bool;

// Derive k challenge vectors of length n from a seed (prod: keccak(beacon || ...)).
pub fn derive_challenges_keccak(seed: &[u8], n: usize, k: usize) -> Vec<Vec<u64>>;
```

The reduction `to_field` uses `rem_euclid` so signed `int8` weights and activations map correctly into $[0, p)$. The module ships seven unit tests covering an honest $2 \times 2$ product, a single tampered entry being caught, negative-entry reduction, deterministic challenge derivation, a fail-closed shape mismatch, a $16{\times}24 \cdot 24{\times}16$ product where flipping one deep entry off-by-one is still caught (the sparse-cheat case that motivates challenging every layer), and a *cross-language golden-parity* test: `derive_challenges_keccak` is asserted against a pinned hex value the Go verifier (`crypto/poi`) emits and asserts in turn, so a Rust-emitted challenge is guaranteed to reproduce on the chain side. The Go mirror carries 22 test functions across its `freivalds`, `wire`, `transcript`, `scale`, and `adversarial` suites.

**The challenge must come after the commitment.** Soundness in Theorem 1 assumes $r$ is unpredictable to the prover. If the prover knew $r$ in advance it could craft $A, B, C$ with $D\,r = 0$ yet $C \neq A \cdot B$. The production derivation therefore draws the seed from an on-chain beacon *after* the prover has committed (§6); `derive_challenges_keccak` is the reproducible expansion both sides apply to that seed, computing for each $(j, i)$ the field element $\text{keccak}(\text{seed} \,\|\, \text{BE}_{32}(j) \,\|\, \text{BE}_{64}(i))[0{:}8] \bmod p$. The unit tests additionally exercise a `splitmix64` stream for the same reproducibility — identical soundness, only the unpredictability source differs.

## 3 Four proofs over one primitive

PoAI is not four mechanisms; it is one matmul-verification primitive applied to four `workloadType`s. The operands $(A, B, C)$ and the I/O commitment differ by workload, but the verifier, the field, and the soundness argument are shared.

**Proof of Embedding (PoE).**
The embedding tower (and any ViT/audio encoder feeding it) is a stack of GEMMs producing a vector. Freivalds verifies the projection layers; the input commitment fixes what was embedded. Settlement uses the embedding as the unit of paid work.

**Proof of Inference (PoI).**
A decode step is the attention and MLP/expert GEMMs plus the output head. The transcript commits the per-layer operands; a challenged layer is opened and Freivalds-checked. This is the canonical case and the one the modal-guess attacker cannot fake.

**Proof of Training (PoT).**
A training step is a forward pass plus a backward pass; the backward pass is again matmuls (the transposed GEMMs of the gradient). The same primitive verifies that a claimed gradient update corresponds to real backpropagation, not a fabricated delta.

**Proof of Compute (PoC).**
The general case: any committed GEMM workload (a kernel an operator claims to have run for hire) is verifiable by the identical $A\,(B\,r) = C\,r$ check. PoE/PoI/PoT are PoC specialized by what the operands mean and what I/O is bound.

This is deliberate orthogonal design: one primitive at one slot, four payoffs. A network that wants to pay for embeddings, inference, training, or raw compute does not need four verifiers — it needs one, parameterized by `workloadType`.

**Settlement: closed-form expert weights from PoAI.** Once work is *verified*, it can be *weighted*. For each expert (operator) $m$, let $q_m \in [0,1]$ be the Bayesian reliability estimated from its historical attestations. Under a product-of-experts combination the optimal mixing weight is closed-form,

$$\eta_m \;\propto\; \frac{q_m}{1-q_m}, \tag{1}$$

i.e. precision-weighted aggregation, with no manual tuning. At decode time this is the product-of-experts rule

$$\log \pi_*(y \mid x) \;=\; \eta_0 \log \pi_0(y \mid x) \;+\; \sum_{i=1}^{K} \eta_i \log \phi_i(y \mid x) \;-\; \log Z, \tag{2}$$

where $\pi_0 = p_\theta$ is the base model, $\{\phi_i\}$ are expert factors, and $Z$ is the per-token partition function. PoAI supplies the reliabilities $q_m$ that make Eq. (1) trustworthy: a weight derived from *verified* work, not from self-reported agreement.

# 4 From matmuls to a graph: typed composition

Theorem 1 secures one matmul. A forward pass is many matmuls glued by non-matmul operations (residual adds, activations, integer rescales, MoE routing). A prover could commit *correct* matmuls computed on *unrelated* inputs — a valid-looking transcript that is not a real forward pass on the actual prompt. PoAI closes this with a typed computation trace (the `poi_graph` module, nine tests), lifting per-op soundness to whole-graph soundness.

**Definition 1** (Graph trace)**.** *A* graph trace *is a finite sequence of typed operations $o_1, \ldots, o_L$ over hash-identified activations. Each $o_\ell$ commits the tuple* $\big($*kind* $\|$ *input commitments* $\|$ *params* $\|$ *output commitment*$\big)$*, where every commitment is a* keccak *digest of the exact-integer activation tensor (with domain separation `DOM_ACT` for activations and `DOM_OP` for op leaves).*

Two checks compose. A trace is *well-formed* (chaining) if every op's input commitment is the output commitment of a prior op or the trace's declared input; a break — an op reading an activation no prior op produced — is visible from the committed leaves alone. A trace is *locally correct* if, when opened, each op recomputes its output from its inputs (a `MatMul` via Freivalds; a deterministic glue op — residual add, activation, integer scale, MoE top-$k$ route, expert gather, weighted combine — by direct exact recomputation) and the result's commitment equals the committed output.

**Theorem 2** (Graph soundness)**.** *If a graph trace is well-formed and every op is locally correct, then the committed final output equals the deterministic forward pass applied to the committed input.*

*Proof.* Induct over a topological order of the ops. Base: the declared input commitment is the true input by definition. Step: consider $o_\ell$ in order. By well-formedness each of its input commitments is either the declared input or the output commitment of some $o_j$, $j < \ell$; by the inductive hypothesis those equal the true intermediate values of the deterministic forward pass. By local correctness $o_\ell$'s committed output is the exact function of its committed inputs that the op computes. Hence $o_\ell$'s committed output equals the true value of the forward pass at that node. Taking $\ell = L$ gives the claim. Everything is exact integer, so equality of commitments is equality of values with zero false-reject. □

The practical payoff: attention's $QK^\top/AV$ and a full MoE block (router → top-$k$ → gather → expert matmuls → combine) are expressible in the trace with *no* floating-point reduction, so the glue is exactly recomputable and the only probabilistic step is Freivalds on the GEMMs.

## 5 Determinism is the load-bearing property

Exact Freivalds needs an exactly reproducible $C$. If two honest runs of the same model on the same input could produce different outputs, there would be no canonical $C$ to commit to, and an honest prover could be false-rejected. Exact verification is therefore *impossible* without bit-level determinism. Two sources of nondeterminism are silently present in ordinary inference, and both are closed in the shipped engine.

1. **Argmax ties.** Greedy decode takes arg max over logits; when two logits are equal, the winner is implementation-defined (it depends on reduction order on the device). The engine pins a **lowest-id tie-break**: among tied maxima the smallest token index wins, on every backend.
2. **MoE expert selection.** A Mixture-of-Experts router picks the top-$k$ experts by score; ties and unstable sort order change *which experts run*, changing $C$ entirely. The engine pins a **canonical top-$k$**: sort by score descending, breaking ties by expert index ascending (the `deterministic_topk_indices` selection).

**Remark 1.** *Determinism is not a performance feature here; it is the precondition for soundness. Only because the int8 accumulator and these two tie-breaks are reproducible can the verifier assert that an honest operator's $C$ matches bit-for-bit, and hence that* any *Freivalds failure is real cheating rather than hardware drift. Proof-bearing runs additionally set `set_moe_proof_deterministic(true)` and pin temperature-0 greedy decode.*

## 6 Commit now, verify later

A forward pass over a large model at hundreds of tokens produces terabytes of activations; retaining them is infeasible, and forcing synchronous verification would serialize the network. PoAI instead has the operator *commit* to its activations cheaply during execution and be *challenged* later on a small random part — the optimistic bargain: cheap in the happy path, partial re-execution only under challenge.

**ActivationTraceMMR.** A streaming **Merkle Mountain Range** accumulates one leaf per $(\text{layer}, \text{token}, \text{tap})$ in execution order and keeps only $O(\log N)$ peak hashes — a frontier of roughly 768 bytes. Activations are hashed as the kernel produces them and then dropped. The hash is `keccak`, with leaf/node domain separation:

$$\begin{aligned} \text{leaf} &= \text{keccak}(\text{DOMAIN\_POI\_ACT} \,\|\, \text{layer} \,\|\, \text{token} \,\|\, \text{tap} \,\|\, \text{dtype} \,\|\, \text{shape} \,\|\, \text{tensor bytes}), \\ \text{node} &= \text{keccak}(\text{DOMAIN\_POI\_NODE} \,\|\, \text{left} \,\|\, \text{right}), \end{aligned}$$

where tap $\in \{$`matmul_in`, `matmul_out`, `block_out`$\}$ — exactly the operands a Freivalds check (or a non-matmul recompute) will open, no more. Lone nodes on odd levels use **RFC-6962 promotion**, *not* Bitcoin duplicate-last: the duplicate-last construction is the CVE-2012-2459 malleability (a forged tree can collide a root), so it is deliberately avoided. The MMR closes by binding the leaf count, $\text{activationTraceRoot} = \text{keccak}(\text{DOMAIN\_POI} \,\|\, n_{\text{leaves}} \,\|\, \text{mmr root})$, so a subtree cannot be reinterpreted as the whole tree.

**Zero overhead when off.** The commitment is threaded as an `Option<&mut ProofTranscript>` through the forward context, not a boolean checked in hot loops. When the option is `None`, each commit site is a single not-taken branch — no hashing, no allocation, no tensor copy. When it is `Some`, the per-matmul hook commits the int8 activation view and the i32 accumulator by borrow (no copy); the weight $B$ is committed once at load into the weights root, never re-hashed per token.

**Challenge, open, bisect.** A verifier derives a layer index $\ell$ and challenge $r$ from a beacon *after* commitment, opens that layer's $A$ (`matmul_in`), $B$ (a reference under the weights root), and $C$ (`matmul_out`) with $O(\log N)$ MMR inclusion proofs, and runs *windowed* Freivalds over a random token sub-range to keep the opened payload small. The verifier recomputes the leaf hashes, checks the inclusions against the committed roots, re-derives $r$, and checks $A\,(B\,r) = C\,r$ — total cost $O(tk + kn + tn + \log N)$, about $\min(t, k, n)\times$ cheaper than recomputing the layer. A mismatch slashes the operator. To localize a *sparse* cheat (a single bad layer among many) the protocol runs interactive **bisection** to the first divergent layer in $O(\log L)$ rounds, so soundness is independent of how many layers are sampled.

# 7 Binding: a proof points at exactly one (model, input, output)

A correct matmul check is worthless if the attestation can be replayed, spliced across tasks, or pointed at a different model. PoAI binds the proof with two hash chains whose constants match the chain side byte-for-byte. The challenge is derived from immutable task context:

$$\text{challenge} = \text{keccak}\big(\text{DOMAIN\_CHALLENGE} \,\|\, \text{taskId} \,\|\, \text{intentID} \,\|\, \text{modelSpecHash} \,\|\, \text{promptHash} \,\|\, \text{openBlockHash} \,\|\, \text{operator}\big),$$

and the attested result commits to that challenge:

$$\text{reportData} = \text{keccak}\big(\text{DOMAIN\_REPORT} \,\|\, \text{challenge} \,\|\, \text{modelSpecHash} \,\|\, \text{promptHash} \,\|\, \text{outputHash} \,\|\, \text{runtimeMeasurement}\big).$$

The chain consumes `reportData`; because it folds in the challenge (which already folds in `taskId`, `intentID`, and `openBlockHash`), a transcript is task-scoped and cannot be reused. This `reportData` is recomputed on-chain from the receipt's own merkle-proven fields by `ComputeProofLib.expectedReportData` and re-checked by the verifier in `AIMiner.mine` (§16), so the binding is enforced at the mint, not merely asserted in prose.

**`modelSpecHash` is measured, not labelled.** The model identity is a keccak Merkle root over the *quantized weight bytes the engine actually loaded* (the per-layer `QuantizedSerde` serialization, with the quant-type discriminant in each leaf). It is not a name string. Re-quantizing the same weights to a different scheme yields a different root; a genuinely different model yields a different root. Combined with a `runtimeMeasurement` that folds in the kernel build id, reduction mode, and sampler/seed, the decoding regime becomes part of identity: the same weights decoded at temperature 2.0, or under a different reduction order, are rejected. The full transcript root chains these together,

$$\text{transcriptRoot} = \text{keccak}\big(\text{DOMAIN\_POI} \,\|\, \text{modelID} \,\|\, \text{inputCommitment} \,\|\, \text{quantizationSpec} \,\|\, \text{kernelVersion} \,\|\, \text{activationTraceRoot} \,\|\, \text{outputCommitment}\big),$$

so the binding runs challenge → model → input → output and an attestation cannot be detached from any link in the chain.

## 8 Floating point: by tier, not by tolerance

The natural temptation is to run Freivalds directly on floating-point GEMMs with a tolerance: accept if $\|A(Br) - Cr\| < \varepsilon$. This is both **unsound** and **false-rejecting**, and PoAI rejects it.

- *Unsound:* a tolerance corridor of width $\varepsilon$ is an attacker's budget. A cheater can perturb $C$ anywhere inside the corridor — a non-trivial space of fabricated outputs — and pass. The clean $1/p$ soundness of Theorem 1 holds only in an exact field.
- *False-rejecting:* floating-point reduction order is not fixed across hardware, so two honest runs differ in the low bits. Any $\varepsilon$ small enough to be sound is small enough to reject honest operators on a different GPU.

PoAI therefore selects the verification regime by numeric tier:

Table 1: Verification regime by numeric tier.

| **Tier** | **Mechanism** | **Soundness basis** |
|---|---|---|
| `int8` / quantized | exact Freivalds over $\mathbb{F}_p$ | software-only, zero false-reject |
| fp16 / bf16 / fp8 | deterministic-fp (RepOps): pin reduction order | bit-reproducible by pinned reduction; or TEE/CC attestation |
| small / niche | zkML circuit | succinct cryptographic proof |

For the quantized path the check is exact and software-only — no special hardware, the common case for the deployed zen models. For native low-precision floating point, the fix is not a tolerance but **determinism**: pin the reduction order (RepOps / deterministic-fp) so the GEMM is bit-reproducible, then the exact equality check applies again; where that is impractical, fall back to a trusted-execution / confidential-compute attestation of the kernel. For small or unusual workloads a zkML circuit gives a succinct proof directly.

**Remark 2.** *A floating-point value* is *an integer — a sign bit, an exponent, and a mantissa. A GEMM executed in a* pinned *reduction order is therefore an exact, reproducible computation over those integers; "deterministic-fp" is not an approximation but a commitment to a specific, replayable order of exact operations. The unsoundness of naive fp-Freivalds comes entirely from* unpinned *order, not from floating point being inherently fuzzy.*

**Coverage across the zen families.** The matmul primitive plus two extensions — a *routing proof* (commit router logits and selected expert indices, verify the canonical top-$k$ chose the same experts, then Freivalds the selected experts only) and *sequential state-chaining* (bind step $t$'s output to step $t{+}1$'s input in the commitment) — covers dense transformers, Mixture-of-Experts, linear-recurrent/SSM/Gated-DeltaNet, multimodal ViT+audio towers, and diffusion/Transfusion. The recurrent chaining serves both linear recurrence and diffusion, so there is no separate diffusion verifier: one primitive, two extensions, full coverage.

## 9 Post-quantum by construction

PoAI's soundness does not rest on any computational hardness assumption, so it does not degrade under a quantum adversary.

**Proposition 3** (Information-theoretic soundness)**.** *Conditioned on the challenge $r$ being drawn uniformly and independently of the prover's committed $(A, B, C)$, the bound $\Pr_r[\, A(Br) = Cr \mid C \neq AB \,] \leq 1/p$ of Theorem 1 holds against a computationally unbounded prover.*

*Proof.* The proof of Theorem 1 is a counting argument over the finite set $\mathbb{F}_p^n$: the bad event is confinement of $r$ to a fixed hyperplane, of density $1/p$, regardless of the prover's running time. No problem is assumed hard. □

The only cryptographic ingredient is the commitment/transcript hash. We instantiate it with keccak (modelled as a random oracle for the domain-separated inputs of §6–7); its collision and preimage resistance against a quantum adversary degrade only by the generic Grover/BHT factors ($2^{128}$ preimage, $2^{85}$ collision at a 256-bit output), which is why the transcript binds at a 256-bit width. The unpredictability of $r$ comes from an on-chain beacon, not from a trapdoor. Consequently the part of the system an attacker would target with a quantum computer — forging a proof of work never done — is governed by Proposition 3, which a quantum computer does not help with.

**Post-quantum at the signing and key-exchange layers.** Where the surrounding protocol does use public-key cryptography — authenticating operators, sealing prompts, carrying attestations across chains — it uses post-quantum primitives, so the *envelope* matches the information-theoretic *core*: ML-DSA (FIPS 204, parameter sets 44/65/87) and SLH-DSA (FIPS 205) for signatures exposed as precompiles; an X-Wing hybrid KEM (X25519 + ML-KEM-768) for prompt sealing, implemented as a pure-Go HPKE library rather than a precompile; the P3Q verifier precompile at slot `0x012205`; and `luxfi/warp` carrying an `MLDSACertSet` so cross-chain attestations are quantum-safe end to end. The network's security claim is thus uniform: *public, leaderless, decentralized, operator-safe, and nation-state-proof*, with no classical-only link in the chain from execution to settlement.

## 10 Implementation status

We state plainly what exists, what is enforced, and what is not yet live.

Table 2: Component status. "Proven+tested" means shipped with green tests; "enforced" means the chain rejects non-compliant claims; "designed" means the spec is merged but no running model emits it yet.

| Component | Where | State |
|---|---|---|
| Rust Freivalds core (prover mirror) | `hanzo-engine/src/poi/` | Proven + tested (7 tests green) |
| Typed graph composition | `poi_graph.rs`, `poi_transcript.rs` | Proven + tested (9 + 8 tests; Thm. 2) |
| Go Freivalds verifier (chain-consumed) | `luxfi/crypto/poi` | Proven + tested (22 test funcs; golden parity) |
| Determinism prerequisites | `hanzoai/engine` | Shipped (lowest-id argmax; canonical MoE top-$k$) |
| No-proof-no-mint enforcement | `AIMiner.sol` | Enforced (fail-closed; 12 forge tests) |
| Bonded supply / slashing | `MinerStakeRegistry.sol` | Enforced (8 forge tests) |
| Network-wide no-double-mint | `AIComputeRegistry.sol` | Enforced (6 forge tests) |
| Capped halving emission | `AICoin.sol` | Enforced (11 forge tests) |
| End-to-end proof *emission* from a running zen model | forward-pass commitment slice | **Designed + merged, NOT yet live** |

To be unambiguous: the verification core and the typed graph composition are **proven and tested** in both Rust (the engine prover mirror) and Go (the chain verifier), the determinism prerequisites are shipped, and the chain **enforces** that no mint occurs without a proof. What is *not* yet live is the end-to-end emission: threading the `ActivationTraceMMR` and the per-layer commit hooks through the engine's forward pass so that a real zen model (e.g. an `int8` MoE) produces a transcript that a challenger opens and verifies. That slice is designed and its wire constants are merged, but no running model emits a proof today. The first true "live" is a green end-to-end test in which an honest int8 zen inference emits a transcript, a beacon-derived layer is challenged, the open verifies — and a faked, cheap-model, or mis-routed run is caught. That is the next slice, not a present claim.

---

PART II — THE PRICE

## 11 The market for verified compute

Part I makes a *proven-compute unit* (PCU) a well-defined good: one unit of AI work that has been Freivalds-verified, bound to a measured model and input, and not double-claimed. Part II prices it. We want a market maker — an always-on counterparty that quotes a price for PCUs against AICoin — with four properties:

1. **No oracle.** The price is endogenous, a function of the maker's own state, not of an external feed (which would reintroduce a trusted party).

2. **Path independence.** The cost of acquiring or releasing inventory depends only on the start and end states, not on the order or fragmentation of the trades; otherwise the maker leaks value to order manipulation.

3. **Bounded subsidy.** Like Hanson's Logarithmic Market Scoring Rule (LMSR), the maker may run at a loss to bootstrap a thin market, but that loss is bounded by a constant the maker chooses in advance.

4. **Supply coupling.** The price must rise with the supply of *verified* compute and be insensitive to unverified claims — the link to PoAI.

We obtain all four from a single structural choice: the market's price one-form is *exact*, i.e. it is the differential of a scalar potential we call the Hamiltonian. This is the abstraction that contains LMSR and constant-function AMMs as special cases and that yields the bounds above as theorems rather than parameters.

# 12 The Hamiltonian formulation

## 12.1 State, potential, price

We begin scalar; §15 lifts to a vector of heterogeneous compute classes. The market state is a point $(q, p)$ in phase space $\Gamma = \mathbb{R} \times \mathbb{R}_{>0}$, where $q \in \mathbb{R}$ is the maker's net inventory of PCUs (positive: the maker has sold PCUs into the market and holds the AICoin; we let $q$ range over $\mathbb{R}$ so the maker can be long or short) and $p \in \mathbb{R}_{>0}$ is the marginal AICoin price of one PCU. We equip $\Gamma$ with the canonical symplectic form $\omega = \mathrm{d}p \wedge \mathrm{d}q$ and the Poisson bracket

$$\{F, G\} \;=\; \frac{\partial F}{\partial q}\frac{\partial G}{\partial p} - \frac{\partial F}{\partial p}\frac{\partial G}{\partial q}. \tag{3}$$

The maker is specified by a single object: a *cost potential*.

**Definition 2** (Cost potential)**.** *A* cost potential *is a function $C : \mathbb{R} \to \mathbb{R}$ that is $C^2$, strictly convex ($C''(q) > 0$ for all $q$), with $C(0) = 0$, and superlinear ($C(q)/|q| \to \infty$ as $|q| \to \infty$, so the price range $C'(\mathbb{R})$ is all of $\mathbb{R}$ and the conjugate below is finite everywhere with $(C^*)' = (C')^{-1}$). The* marginal price *the maker quotes at inventory $q$ is*

$$p(q) \;=\; C'(q), \tag{4}$$

*and the* cash a trader pays *to move the maker's inventory from $q_0$ to $q_1$ is the path integral of the price one-form,*

$$\mathrm{Cost}(q_0 \to q_1) \;=\; \int_{q_0}^{q_1} p\,\mathrm{d}q \;=\; C(q_1) - C(q_0). \tag{5}$$

Strict convexity makes $p(q) = C'(q)$ strictly increasing, hence invertible; write its inverse (the *inventory at price $p$*) as $q(p) = (C')^{-1}(p)$. Buying PCUs from the maker decreases the maker's inventory and (since $C'' > 0$) *raises* the marginal price; selling does the reverse. This is the AMM slippage law, here derived from convexity.

### 12.2 The Hamiltonian and the equations of motion

The conserved quantity — the "energy" invariant along a trade — is the maker's *bonded value*: the AICoin it must escrow to back its quotes. We define it as the Legendre (convex) conjugate of the cost potential.

**Definition 3** (HMM Hamiltonian)**.** *Let $C$ be a cost potential with conjugate $C^*(p) = \sup_{q\in\mathbb{R}}\{pq - C(q)\}$. The* Hamiltonian *is the bonded value as a function of state,*

$$H(q,p) \;=\; p\,q - C(q). \tag{6}$$

On the *quote manifold* $\Sigma = \{(q,p) : p = C'(q)\}$ — the states the maker actually occupies — the supremum in the conjugate is attained at the operating $q$, so $H$ restricted to $\Sigma$ equals the conjugate evaluated at the quoted price:

$$H\big(q, C'(q)\big) \;=\; C'(q)\,q - C(q) \;=\; C^*\big(C'(q)\big), \qquad q(p) = (C^*)'(p). \tag{7}$$

The second identity is the standard Legendre relation: the inventory is the derivative of the conjugate potential. Equation (7) is the heart of the construction — it is what makes $H$ a clean function of price alone on $\Sigma$.

Two distinct motions live on $\Gamma$, and it is worth separating them cleanly. A *trade* is a controlled displacement *along* the quote manifold $\Sigma$: the trader names a new inventory and pays the potential difference (5); the maker stays on its own curve $p = C'(q)$ throughout, so a trade is parametrized by inventory, not by a clock. *Arbitrage relaxation* is the off-$\Sigma$ motion that restores the quote when an external price diverges from it, and *this* is what the Hamiltonian flow describes. Hamilton's equations with respect to (3) are

$$\dot q \;=\; \{q, H\} \;=\; \frac{\partial H}{\partial p} \;=\; q, \qquad \dot p \;=\; \{p, H\} \;=\; -\frac{\partial H}{\partial q} \;=\; -\big(p - C'(q)\big). \tag{8}$$

The $\dot p$ equation is the meaningful one: it is a *restoring force toward the quote manifold.* Off $\Sigma$ (where $p \neq C'(q)$) it pushes the price back to the quoted value $C'(q)$; on $\Sigma$ the restoring term vanishes and the maker rests. (The $\dot q = q$ equation is a parametrization artifact of the bonded-value generator and carries no economic content — inventory is moved by trades along $\Sigma$, Eq. (5), not by this flow time; this is why we never integrate $\dot q$ to draw an economic conclusion.) The arbitrage gap

$$\Phi(q,p) \;=\; p - C'(q) \tag{9}$$

is therefore the natural "displacement" coordinate, with $\Sigma = \{\Phi = 0\}$ its equilibrium. The invariant of the relaxation that matters is the bonded value $H$ itself: along $\Sigma$ it equals the collateral $C^*(p)$ (Eq. (7)), a state function, so no closed trade cycle changes it (Thm. 5). We will see that $\Sigma$ is exactly the no-arbitrage manifold.

**Remark 3** (Why this $H$, and not a hand-built energy)**.** *One could write down many Hamiltonians; only one earns the name here. The bonded value $H = pq - C(q)$ is forced by two requirements: (i) its $q$-gradient must vanish on the quote manifold (so the maker is at rest exactly when it quotes its own cost curve), which forces $\partial_q H = -(p - C'(q))$ and hence $H = pq - C(q) + h(p)$; and (ii) on $\Sigma$ it must equal the collateral the maker has to hold, the conjugate $C^*(p)$, which by* (7) *forces $h \equiv 0$. There is no free functional choice left. This is the discipline the earlier "harmonic potential" formulations lacked: a term added to $H$ for stability that has no collateral interpretation also has no conservation theorem behind it.*

# 13 Conservation laws and the four guarantees

## 13.1 Path independence

**Theorem 4** (Path independence)**.** *For any cost potential $C$, the cash a trader pays to move the maker between two inventory levels depends only on the endpoints:*

$$\mathrm{Cost}(q_0 \to q_1) \;=\; C(q_1) - C(q_0), \tag{10}$$

*independently of how the move is decomposed into intermediate trades. Equivalently, the price one-form $p\,\mathrm{d}q = C'(q)\,\mathrm{d}q = \mathrm{d}C$ is exact.*

*Proof.* By Eq. (4), $p\,\mathrm{d}q = C'(q)\,\mathrm{d}q = \mathrm{d}C(q)$, an exact one-form. For any partition $q_0 = x_0, x_1, \ldots, x_m = q_1$ the total cost telescopes: $\sum_{j=1}^{m} \big(C(x_j) - C(x_{j-1})\big) = C(q_1) - C(q_0)$. Because the integrand is a total differential, the line integral (5) is independent of the intermediate points and of any subdivision. □

Path independence is the property that defeats trade-splitting and ordering games: a trader who breaks one large order into many, or interleaves buys and sells, pays exactly what a single net trade would cost. It is the discrete-market analogue of a *conservative force field* in mechanics, and it holds for *every* HMM because every HMM is built from a potential.

## 13.2 Bounded round-trip and no in-curve arbitrage

**Theorem 5** (Zero-cost round-trip; no arbitrage)**.** *Let the maker charge a proportional fee at rate $\gamma \geq 0$ on the gross cash of each trade. Then:*

*(a) With $\gamma = 0$, any closed trajectory (one returning the maker to its starting inventory $q_0$) has net trader cost exactly 0.*

*(b) With $\gamma > 0$, any closed trajectory that involves at least one trade of nonzero size has net trader cost strictly positive; equivalently, no trader can extract value by a round trip against the curve.*

*Proof.* (a) By Theorem 4 the fee-free cost of $q_0 \to \cdots \to q_0$ is $C(q_0) - C(q_0) = 0$; the conservative one-form integrates to zero around any loop. (b) The fee on a single trade $x_{j-1} \to x_j$ is $\gamma\,|C(x_j) - C(x_{j-1})| \geq 0$, and is strictly positive whenever $x_j \neq x_{j-1}$ because $C$ is strictly monotone in price and hence injective. The net cost of the loop is the fee-free part (zero, by (a)) plus $\gamma \sum_j |C(x_j) - C(x_{j-1})| > 0$ as soon as one step is nontrivial. □

**Corollary 6** ($\Sigma$ is the no-arbitrage manifold)**.** *A price $p$ admits no instantaneous arbitrage against the maker if and only if $p = C'(q)$, i.e. $(q, p) \in \Sigma$ (the gap $\Phi = 0$). Off $\Sigma$, buying at the quoted $C'(q) < p$ and reselling at $p$ (or the reverse) yields a strictly positive profit equal to $|p - C'(q)|$ per unit; on $\Sigma$ that profit is zero.*

*Proof.* The marginal price the maker quotes is $C'(q)$ by Eq. (4). If an external price $p \neq C'(q)$ exists, the gap $\Phi = p - C'(q) \neq 0$ is exactly the per-unit profit of trading with the maker and against the external price; arbitrage exists. If $p = C'(q)$ the gap is zero and no costless profit exists. The dynamics (8) drive $\Phi \to 0$, i.e. arbitrage closes the gap and restores $\Sigma$, consistent with the restoring force interpretation. □

This is the economic content of energy conservation: the Hamiltonian flow has no "free energy" to give away around a cycle, so the only way the maker pays out is through deliberate subsidy (next) and the only way a trader profits from a cycle is by supplying fees.

### 13.3 Bounded market-maker subsidy

A market maker that quotes a finite price for the first unit of a brand-new market necessarily subsidizes price discovery: early trades move inventory cheaply, and if the "true" price later settles far away the maker realizes a loss. Hanson's LMSR famously bounds this loss by a constant. The HMM bounds it by the *range of the conjugate potential*, and LMSR's bound falls out as the entropic instance.

**Theorem 7** (Bounded subsidy)**.** *Suppose the market opens at inventory $q_0$ with $C(q_0) = 0$ (the maker has taken in no cash yet) and that admissible inventories are confined to a set $\mathcal{Q} \subseteq \mathbb{R}$ on which $C$ is bounded below by $C_{\min} = \inf_{q \in \mathcal{Q}} C(q)$. Then over* any *sequence of trades that ends at some terminal inventory $q_T \in \mathcal{Q}$, the maker's net payout (cash out minus cash in) is at most*

$$\text{Subsidy} \;=\; C(q_0) - C(q_T) \;\le\; -\,C_{\min} \;=\; C(q_0) - C_{\min}. \tag{11}$$

*In particular the worst-case subsidy is the constant $C(q_0) - C_{\min}$, independent of the path and of the number of trades.*

*Proof.* By Theorem 4 the total cash the maker has *received* from traders between open and close is $C(q_T) - C(q_0)$ (negative when it has paid out on net). The maker's net payout is the negative of its receipts, $C(q_0) - C(q_T)$. Since $q_T \in \mathcal{Q}$, $C(q_T) \ge C_{\min}$, hence $C(q_0) - C(q_T) \le C(q_0) - C_{\min}$. With $C(q_0) = 0$ this is $-C_{\min}$. The bound references only endpoints, so it is path- and count-independent. □

**Corollary 8** (LMSR is the entropic HMM, with subsidy $b \ln N$)**.** *Let the market have $N$ exclusive outcomes with inventory vector $\boldsymbol{q} \in \mathbb{R}^N$ and take the entropic cost potential (the log-partition function)*

$$C(\boldsymbol{q}) \;=\; b\,\ln \sum_{i=1}^{N} e^{q_i/b}, \qquad b > 0, \tag{12}$$

*whose gradient $p_i = \partial C/\partial q_i = e^{q_i/b}/\sum_j e^{q_j/b}$ is the LMSR price (a probability on the simplex). Open at $\boldsymbol{q}_0 = \mathbf{0}$, so $C(\boldsymbol{q}_0) = b \ln N$. At resolution to outcome $i^\star$ the maker pays out $q_{i^\star} \le \max_i q_i$ per share held against the cash $C(\boldsymbol{q}) - C(\mathbf{0})$ it took in. Then the maker's worst-case net loss is the constant*

$$\text{Subsidy}_{\max} \;=\; \sup_{\boldsymbol{q}} \Big( \max_i q_i - \big(C(\boldsymbol{q}) - C(\mathbf{0})\big) \Big) \;=\; b \ln N, \tag{13}$$

*recovering Hanson's bound exactly.*

*Proof.* $C(\mathbf{0}) = b \ln \sum_i e^0 = b \ln N$. The maker's net loss at state $\boldsymbol{q}$ is the payout minus the receipts, $\max_i q_i - \big(C(\boldsymbol{q}) - C(\mathbf{0})\big)$. Since $C(\boldsymbol{q}) = b \ln \sum_i e^{q_i/b} \ge b \ln e^{\max_i q_i/b} = \max_i q_i$, we have $\max_i q_i - C(\boldsymbol{q}) \le 0$, with the bound approached ($\to 0$) as one coordinate dominates. Hence the supremum of the net loss equals $C(\mathbf{0}) = b \ln N$. This is the standard LMSR worst-case-loss computation, here read off the general HMM bound (11): $C(\mathbf{0}) = b \ln N$ is the open value and $\sup(\max_i q_i - C) = 0$ plays the role of $-C_{\min}$. □

Corollary 8 is the payoff of putting LMSR inside the Hamiltonian frame: its celebrated bounded loss is not special to the logarithm, it is the general HMM subsidy bound (11) instantiated at the entropic potential. A compute market chooses the liquidity parameter $b$ (or, in the scalar compute case, the curvature of $C$) to trade off depth against the constant subsidy it is willing to spend to bootstrap price discovery.

### 13.4 The monotone price–supply law (the PoAI coupling)

**Theorem 9** (Monotone price–supply law)**.** *Let $s \geq 0$ denote the cumulative supply of* verified *PCUs the maker has absorbed (each a distinct, Freivalds-verified, non-double-claimed unit; see §16), and let inventory track supply, $q = q_0 - s$ (absorbing supply lowers the maker's net short position). Then the quoted price $p(s) = C'(q_0 - s)$ is strictly decreasing in $s$ on the buy side and, dually, the price a* consumer *pays per PCU is strictly increasing in demand pressure; in both cases the sensitivity is governed by the curvature:*

$$\frac{\mathrm{d}p}{\mathrm{d}s} \;=\; -\,C''(q_0 - s) \;<\; 0, \qquad \left|\frac{\mathrm{d}p}{\mathrm{d}s}\right| \;=\; C''(q) \;>\; 0. \tag{14}$$

*Consequently the price moves* only *in response to changes in* verified *supply: an unverified claim never reaches $q$ (it is rejected before the inventory update, §16), so $\mathrm{d}p/\mathrm{d}s = 0$ along it.*

*Proof.* $p(s) = C'(q_0 - s)$; differentiate, $\mathrm{d}p/\mathrm{d}s = -C''(q_0 - s) < 0$ by strict convexity, with magnitude $C''(q)$. The second clause is structural rather than analytic: the inventory update $q \mapsto q - 1$ that drives the price is executed by the mint path *only after* the PoAI compute proof and the no-double-mint claim succeed (Eq. (19) below), so a claim that fails verification leaves $q$, and hence $p$, unchanged: $\mathrm{d}p/\mathrm{d}s\big|_{\text{unverified}} = 0$. □

Theorem 9 is the formal statement of "the market prices *verified* useful work, not raw tokens." The curvature $C''$ is the single knob: large $C''$ means a deep, slow-moving price (much verified supply needed to move it); small $C''$ means a shallow, responsive price.

### 13.5 Slippage is convex

**Proposition 10** (Convex slippage / quote-width)**.** *The average price paid for a block trade of size $\delta$ starting at inventory $q$,*

$$\bar{p}(q, \delta) \;=\; \frac{C(q + \delta) - C(q)}{\delta}, \tag{15}$$

*is increasing in $\delta$ for $\delta > 0$, and $\bar{p}(q, \delta) \to C'(q)$ as $\delta \to 0$. The realized slippage $\bar{p}(q, \delta) - C'(q)$ is nonnegative and convex in $\delta$.*

*Proof.* $\bar{p}$ is the average of $C'$ over $[q, q + \delta]$; since $C'$ is increasing (strict convexity of $C$), this average increases with $\delta$ and tends to the left endpoint value $C'(q)$ as $\delta \to 0$. Slippage $\bar{p} - C'(q) = \frac{1}{\delta}\int_q^{q+\delta} (C'(u) - C'(q))\,\mathrm{d}u \geq 0$; its convexity in $\delta$ follows from $C'$ increasing (the second difference of $C$ is nonnegative). □

This is the standard depth guarantee: large trades pay strictly more per unit, by an amount the maker controls through $C''$, and the marginal quote is recovered in the small-trade limit.

## 14 LMSR and Uniswap are two potentials of one framework

The Hamiltonian frame is not a new market maker bolted next to the known ones; it is the genus of which they are species. A market maker is an HMM precisely when its price one-form is exact — when it has a potential. We make the two canonical cases explicit.

**LMSR (Hanson): the entropic potential.** Take $C(\boldsymbol{q}) = b \ln \sum_i e^{q_i/b}$ (Eq. (12)). The price is the softmax, $p_i = \mathrm{softmax}(\boldsymbol{q}/b)_i$, a point on the probability simplex; the Hamiltonian $H = \langle \boldsymbol{p}, \boldsymbol{q} \rangle - C(\boldsymbol{q})$ restricted to the quote manifold is the negative Shannon entropy scaled by $b$ (the conjugate of log-sum-exp is the negative entropy on the simplex). Bounded subsidy $b \ln N$ is Corollary 8. This is the right maker for an *outcome* market — which model wins, which answer is correct — where prices must be a probability distribution.

**Uniswap (constant function): the logarithmic-reserve potential.** A constant-function market maker holds reserves $(x, y) \in \mathbb{R}^2_{>0}$ and keeps an invariant $\varphi(x, y) = \kappa$ fixed across every trade. For Uniswap, $\varphi(x, y) = x\,y$, so $H_{\mathrm{cf}}(x, y) = x\,y = \kappa$ is *literally* the conserved Hamiltonian — the constant-function invariant *is* an energy. The marginal price of $x$ in units of $y$ along the curve is $p = -\mathrm{d}y/\mathrm{d}x = y/x$ (differentiate $xy = \kappa$). One can recover these quotes from a scalar generator: parametrizing by $u = \ln x$ and setting $g(u) = -\kappa e^{-u} = -y$ gives $g'(u) = \kappa e^{-u} = y$, so the quoted price is $g'(u)/x = y/x$. We call $x \cdot y = k$ a *degenerate* conserved quantity for two reasons. First, the generator $g$ is concave rather than strictly convex, so it is not a cost potential in the sense of Definition 2; the constant-function maker conserves the invariant $\varphi = xy$ directly, and it is this invariant (not a convex cost) that plays the Hamiltonian role. Second, unlike LMSR's potential, the constant-product invariant is a single hypersurface with no liquidity parameter $b$ separating depth from subsidy — depth and price-impact are locked together by $\kappa$ alone. The shared structure that makes both HMMs is still the same: the price one-form is exact (here $p\,\mathrm{d}x = -\mathrm{d}y$ along $\varphi = \kappa$). This is the right maker for a *fungible-pair* market with two-sided reserves.

Table 3: Two market makers as two potentials in the one Hamiltonian frame. "Exact one-form" is the defining property; the rest follow from §13.

| | **LMSR (Hanson)** | **Uniswap (CFMM)** |
|---|---|---|
| Potential $C$ | $b \ln \sum_i e^{q_i/b}$ (entropic) | $-\kappa e^{-u}$, $u = \ln x$ (log-reserve) |
| Conserved $H$ | $-b\cdot$entropy on $\Sigma$ | $x\,y = \kappa$ (the invariant itself) |
| Price | softmax $\in$ simplex | $y/x$ |
| Liquidity knob | $b$ (separates depth / subsidy) | $\kappa$ only (depth $\equiv$ price-impact) |
| Bounded subsidy | $b \ln N$ (Cor. 8) | divergence loss $2\sqrt{r}/(1+r) - 1$ |
| Natural market | exclusive outcomes | fungible pair |

**Proposition 11** (Constant-function divergence-loss bound)**.** *For the Uniswap potential, an LP whose position experiences a price ratio $r = p_1/p_0$ between deposit and withdrawal suffers value ratio*

$$\frac{V_{pool}}{V_{hold}} = \frac{2\sqrt{r}}{1+r} \leq 1, \tag{16}$$

*with equality iff $r = 1$; the loss $1 - 2\sqrt{r}/(1+r)$ is the constant-function analogue of the bounded*

*subsidy and is itself bounded on any bounded price range.*

*Proof.* Along $xy = \kappa$ with price $p = y/x$ we have $x = \sqrt{\kappa/p}$ and $y = \sqrt{\kappa p}$. The pool value (in the numéraire $y$) is $V_{\text{pool}} = p\,x + y = \sqrt{\kappa p} + \sqrt{\kappa p} = 2\sqrt{\kappa p} = 2\sqrt{\kappa p_0}\,\sqrt{r}$, using $p = r p_0$. Holding the initial basket $(x_0, y_0) = (\sqrt{\kappa/p_0}, \sqrt{\kappa p_0})$ gives $V_{\text{hold}} = p\,x_0 + y_0 = r p_0\sqrt{\kappa/p_0} + \sqrt{\kappa p_0} = \sqrt{\kappa p_0}\,(r+1)$. Hence $V_{\text{pool}}/V_{\text{hold}} = 2\sqrt{r}/(1+r)$. By AM–GM $1 + r \geq 2\sqrt{r}$, so the ratio is $\leq 1$ with equality iff $r = 1$. □

The unification is the point: *path independence, no-arbitrage, and a bounded maker loss are not features one bolts onto a market maker; they are automatic consequences of the price one-form being exact.* LMSR and Uniswap are the two potentials practitioners already use; the HMM names the structure they share and lets a compute market choose a potential fit to its own good — which for PCUs is a strictly convex scalar (or weighted multi-asset) $C$ with a tunable curvature.

## 15 Multi-asset HMM over heterogeneous compute

Compute is not one good. A PCU on an H100 at `int8`, a PCU on consumer silicon at `fp16`, a long-context decode, and a training step are different resources with different scarcity. We lift the scalar HMM to a vector of $n$ compute classes with a *separable weighted* potential, which keeps every theorem of §13 and adds cross-class structure only where wanted.

**Definition 4** (Weighted multi-asset HMM)**.** *Let $\boldsymbol{q} = (q_1, \ldots, q_n) \in \mathbb{R}^n$ be inventories of $n$ compute classes with weights $\boldsymbol{w}$, $w_i > 0$, $\sum_i w_i = 1$, and per-class strictly-convex potentials $c_i$. The pool potential is*

$$C(\boldsymbol{q}) \;=\; \sum_{i=1}^{n} w_i\, c_i(q_i), \qquad p_i \;=\; \frac{\partial C}{\partial q_i} \;=\; w_i\, c_i'(q_i), \tag{17}$$

*and the Hamiltonian is $H(\boldsymbol{q}, \boldsymbol{p}) = \langle \boldsymbol{p}, \boldsymbol{q} \rangle - C(\boldsymbol{q})$ with the canonical bracket $\{F, G\} = \sum_i (\partial_{q_i} F\, \partial_{p_i} G - \partial_{p_i} F\, \partial_{q_i} G)$.*

**Proposition 12** (Multi-asset guarantees)**.** *For the weighted multi-asset HMM of Definition 4:*

*(a) (Path independence) $\sum_i p_i\, \mathrm{d}q_i = \mathrm{d}C$ is exact; the cost of moving $\boldsymbol{q}_0 \to \boldsymbol{q}_1$ is $C(\boldsymbol{q}_1) - C(\boldsymbol{q}_0)$, route-independent.*

*(b) (No arbitrage) The no-arbitrage manifold is $\{\boldsymbol{p} = \nabla C(\boldsymbol{q})\}$; off it, the gap $\boldsymbol{p} - \nabla C(\boldsymbol{q})$ is the per-unit arbitrage profit vector.*

*(c) (Bounded subsidy) The worst-case subsidy is $C(\boldsymbol{q}_0) - \inf_{\boldsymbol{q} \in \mathcal{Q}} C(\boldsymbol{q})$, the multi-asset form of* (11)*.*

*(d) (Permutation symmetry / no preferred class) If $w_i = w_j$ and $c_i = c_j$, the potential $C$ is invariant under the inventory swap $\sigma : (q_i, q_j) \mapsto (q_j, q_i)$, so the maker quotes the two classes by the* same *curve: $p_i = p_j$ whenever $q_i = q_j$, and the equal-inventory diagonal $\{q_i = q_j\}$ is invariant under the symmetric two-class flow. No class is systematically cheaper to drain than its identical twin.*

*Proof.* (a)–(c) are the gradient-field versions of Theorems 4, 5, and 7: a separable sum of exact one-forms is exact, and the endpoint accounting is unchanged. For (d): with $w_i = w_j =: w$ and $c_i = c_j =: c$ the summand $w_i c_i(q_i) + w_j c_j(q_j) = w(c(q_i) + c(q_j))$ is symmetric in $(q_i, q_j)$, and the other summands do not involve these two coordinates, so $C \circ \sigma = C$. Hence the quoted prices $p_i = w\, c'(q_i)$ and $p_j = w\, c'(q_j)$ are the *same* function of the respective inventory, giving $p_i = p_j$ exactly when $q_i = q_j$. By Noether's theorem the discrete symmetry $\sigma$ maps trajectories to

trajectories; in particular the fixed-point set $\{q_i = q_j\}$ of $\sigma$ is invariant under the $\sigma$-symmetric flow generated by $H$ (a trajectory starting on the diagonal maps to itself under $\sigma$ and so cannot leave it). Thus the maker has no curve-level bias between identical classes. □

**Remark 4** (Why not an "angular momentum")**.** *One is tempted to claim a conserved* $L_{ij} = q_i p_j - q_j p_i$ *by analogy with rotational symmetry. For the cost-function maker this is* false *in general: a direct computation gives* $\{L_{ij}, H\} = w\,(q_i\,c'(q_j) - q_j\,c'(q_i))$*, which vanishes only on the diagonal* $q_i = q_j$*, not identically. We therefore claim only the genuine discrete permutation symmetry above. A continuous rotational symmetry would require a rotationally invariant* $C$ *(e.g.* $C$ *a function of* $\|\boldsymbol{q}\|$ *alone), which is not the separable potential a heterogeneous compute market wants; importing the conserved* $L_{ij}$ *without that hypothesis is exactly the kind of unforced "physics" the construction is built to avoid (Remark 3).*

**Remark 5** (No spurious dynamics)**.** *We deliberately do* not *add a kinetic term* $\frac{1}{2}\sum_i \mu_i p_i^2$ *to* $H$*. Such a term makes "mass" nonzero and yields oscillatory price dynamics, but those oscillations have no market meaning — a market maker is not a pendulum, and a momentum that carries price past its quote is precisely the value leak path-independence is meant to forbid. The clean conservation laws above hold because* $H$ *is the bonded value and nothing else (Remark 3). Heterogeneity is expressed through the weights* $\boldsymbol{w}$ *and the per-class curvatures* $c_i''$*, which is where it belongs.*

## 16 Binding the curve to verified supply

The HMM's guarantees are only as meaningful as the inventory axis $q$. If $q$ could be incremented by an unverified claim — a modal guess — the maker would price fiction. PoAI is precisely what makes each unit of $q$ a distinct, verified, non-replayable PCU. Three on-chain mechanisms, all read in §§2–10, enforce this, and we state the composite mint gate.

**(1) Bonded supply:** `MinerStakeRegistry`**.** A miner that supplies compute must publicly bond capital in proportion to the capacity it declares. The contract holds, for each miner, `bonded` wei and `capacity` units, and exposes the single read the mint path needs:

$$\texttt{eligible}(m) \iff \neg\texttt{blacklisted}(m) \wedge \texttt{unbondAt}(m) = 0 \wedge \texttt{bonded}(m) \;\geq\; \max\big(\texttt{minBond},\, \texttt{capacity}(m)\cdot\texttt{bondPe} \tag{18}$$

Three authorities are kept orthogonal (one per concern): the *miner* controls its own bond with a withdrawal cooldown; the *slasher* — the compute-proof fraud verifier — slashes the bond on a *proven* discrepancy, no vote; and *governance* may blacklist for off-protocol harm. The cooldown is the fraud window: a miner cannot dodge a fraud proof by exiting first, because `slash` works during the cooldown. Economically, the bond is the supply-side honesty guarantee feeding the curve: producing a PCU puts real capital at risk against the Freivalds check of §2, so the expected value of a fabricated unit is negative.

**(2) No-proof-no-mint:** `AIMiner`**.** The increment of $q$ (the mint of the subsidy that pays for one PCU) is gated, in pure Solidity that runs on any EVM, on a valid `ComputeProof`. The contract fails *closed*: if no verifier is wired it reverts `ProofGateUnavailable`; when wired, it recomputes the expected `reportData` from the receipt's own merkle-proven fields and requires

$$\texttt{verifier.verify}\big(\texttt{computeProof},\, \texttt{ComputeProofLib.expectedReportData}(\dots),\, \texttt{runtimeMeasurement}\big) \;=\; \texttt{tru} \tag{19}$$

A forged receipt root alone (the "C3" hole) no longer mints: inclusion under a root proves the chain committed the receipt, and the compute proof proves the output is real computation. This is the on-chain face of the signature/plurality/computation distinction of §1.

**(3) Network-wide no double mint: `AIComputeRegistry`.** A prover could do the work once and submit distinct, chain-bound proofs of the *same* computation to many chains, minting $N$ times for one unit. The registry keys on the *chain-independent* commitment

$$\texttt{computationKey} \;=\; \mathrm{keccak}\big(\mathrm{DOMAIN} \,\|\, \texttt{modelSpec} \,\|\, \texttt{promptHash} \,\|\, \texttt{outputHash}\big), \tag{20}$$

records the first claimant, and reverts `AlreadyClaimed` for any second claim from any miner on any chain. First-correct-proof wins; redundant recomputation earns nothing. This guarantees the inventory increments are *distinct* units, so $q$ counts each PCU once.

**The composite supply axiom.** Together, (18), (19), and the registry give the property Theorem 9 assumes:

**Proposition 13** (Verified-supply integrity)**.** *Every increment $q \mapsto q-1$ on the buy side of the HMM corresponds to a unit of compute that is (i) produced by a miner satisfying* (18)*, hence bonded and slashable; (ii) accompanied by a `ComputeProof` satisfying* (19)*, hence Freivalds-verified and bound to a measured (`modelSpec`, `promptHash`, `outputHash`); and (iii) claimed once network-wide via a unique `computationKey`. No other transition increments $q$.*

*Proof.* The only path that mints the subsidy (and thereby moves inventory) is `AIMiner.mine`, which in sequence: checks the receipt shape and finality; proves merkle inclusion under an accepted root; requires (19) or reverts (fail-closed); claims the `computationKey` on the registry, reverting on a duplicate; and only then calls `mintSubsidy`, which is itself restricted to contract minters (the god-key defense) and clamped to the halving allowance. Each conjunct of the claim is one of these gates; the absence of any other minter of the coin (enforced by `setMinter` requiring contract code and the single `mintSubsidy` entry point) gives the "no other transition" clause. □

**Coupling to issuance.** The coin the HMM denominates is itself bounded: `AICoin` is a capped, halving, fair-launch subsidy with `MAX_SUBSIDY` $= 10^9$ and a four-year halving period, vesting

$$\texttt{cumulativeAllowance}(t) \;=\; \mathrm{MAX}\big(1-2^{-k}\big)+\big(\mathrm{MAX}\cdot 2^{-(k+1)}\big)\cdot\frac{t \bmod T}{T}, \qquad k=\left\lfloor \tfrac{t}{T} \right\rfloor,\; T = 4\,\text{years}, \tag{21}$$

with $\texttt{emissionAllowance}(t) = \texttt{cumulativeAllowance}(t) - \texttt{mintedSubsidy}$ clamped at zero, so cumulative mint can never exceed the cap. There is a clean duality here: the *issuance* side bounds how much coin can ever enter (Eq. (21), a geometric sum to MAX), and the *market* side bounds how much the maker can ever subsidize price discovery (Eq. (11), the conjugate range). Both are constants chosen in advance; neither is an oracle. The HMM prices PoAI-verified compute into this fixed-supply coin, and every quantity in the loop — the proof, the bond, the double-mint guard, the curve, the cap — is a mechanism we read in the shipped code, not a parameter we wished for.

# 17 Conclusion

A network that pays for AI must do two hard things, and we have done both with one honest seam. It must *verify computation*, not authorship or agreement: the decisive observation is that a forward

pass is almost entirely matmuls, that a matmul can be *verified* an order of magnitude below the cost of *performing* it, and that on the engine's exact integer accumulator this verification is bit-exact with zero false-reject and information-theoretically sound, hence post-quantum. PoAI builds directly on the Hanzo engine: one Freivalds primitive over $\mathbb{F}_p$, specialized into four proofs, lifted to whole-graph soundness by typed composition, made sound by determinism, made asynchronous by a streaming Merkle Mountain Range with interactive bisection, and made unforgeable by a measured-weights binding into `reportData`. And it must *price* that verified work without an oracle: the Hamiltonian Market Maker is the principled generalization of LMSR and constant-function AMMs — a market maker whose conserved bonded value gives path independence, no in-curve arbitrage, and a bounded subsidy as *theorems*, and whose monotone price–supply law ties the AICoin price of compute to the supply of *verified* PCUs. The bond and the no-double-mint registry are what make that supply axis honest; the capped halving coin is what the curve denominates. The proof core, the graph composition, the determinism, and all four supply-side contracts are proven, tested, and enforced today; emitting the proof end-to-end from a running zen model is the next, designed slice. The economics are complete on paper because they are theorems; the systems are complete in code because we cite where they live.

## References


[1] R. Freivalds. Probabilistic machines can use less running time. *IFIP Congress*, 1977.

[2] R. Hanson. Combinatorial information market design. *Information Systems Frontiers*, 5(1):107–119, 2003. (Logarithmic Market Scoring Rule; bounded loss $b \ln N$.)

[3] R. Hanson. Logarithmic market scoring rules for modular combinatorial information aggregation. *Journal of Prediction Markets*, 1(1):3–15, 2007.

[4] H. Adams, N. Zinsmeister, D. Robinson. Uniswap v2 Core. 2020. (Constant-product $x \cdot y = k$.)

[5] G. Angeris, T. Chitra. Improved price oracles: constant function market makers. *ACM Conf. on Advances in Financial Technologies (AFT)*, 2020.

[6] V. I. Arnold. *Mathematical Methods of Classical Mechanics*, 2nd ed. Springer, 1989. (Symplectic geometry; Hamilton's equations; Liouville's theorem.)

[7] R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970. (Legendre–Fenchel conjugacy; $C^*(p) = \sup_q \{pq - C(q)\}$.)

[8] Hanzo AI. `hanzo-engine/src/poi.rs`, `poi_graph.rs`, `poi_transcript.rs`, `poi_forward.rs` — Proof-of-AI Freivalds core and typed graph composition. `hanzoai/engine`.

[9] Lux. `crypto/poi` — canonical Go Freivalds verifier (`NewMat`, `Verify`, `VerifyMulti`, `DeriveChallenges`). `luxfi/crypto`.

[10] Lux. `AIMiner.sol` (`contracts/ai/mining`), `ComputeProofLib.sol`, `ComputeWitnessLib.sol` (`contracts/ai/compute`) — on-chain `reportData` binding and no-proof-no-mint enforcement. `luxfi/standard` (BSD-3-Eco), consumed via `@luxfi/standard/ai/mining`.

[11] Lux. `MinerStakeRegistry.sol` — bonded, slashable compute supply. `luxfi/standard contracts/ai/mining`.

[12] Lux. `AIComputeRegistry.sol` — network-wide no-double-mint computation ledger. `luxfi/standard contracts/ai/compute`.

[13] Lux. `AICoin.sol` — capped halving fair-launch subsidy (MAX = $10^9$, four-year halving). `luxfi/standard contracts/ai/token`.

[14] B. Laurie, A. Langley, E. Kasper. Certificate Transparency. RFC 6962, 2013. (Lone-node promotion; avoids CVE-2012-2459 duplicate-last malleability.)

[15] NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA), 2024.

[16] NIST. FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA), 2024.

[17] M. Barbosa et al. X-Wing: The Hybrid KEM You've Been Looking For. 2024. (X25519 + ML-KEM-768 hybrid; used for prompt sealing.)

[18] Hanzo AI. Hanzo Consensus AI: Consensus Mechanisms for Distributed AI Training and Inference. `hanzoai/papers`, 2026.